

Block Ciphers, AES and Integrity

CSC 3114: Cryptology and Coding Theory, Take-Home Assignment (Test 2)

Submitted: 7 October 2026

Group Members: Group 4

#	Name	Registration No.	Student No.	Tasks
1	NANSEREKO HOUSNAH	24/U/09631/EVE	2400709631	Task 3 (AES Modes)
2	SUUNA RAYMOND	24/U/11403/EVE	2400711403	Task 4 (Integrity), Task 5 (Reflection & Video)
3	NABUKEERA SUMAYAH KASWA	24/U/07899/EVE	2400707899	Task 1 (Theory), Task 2 (Avalanche)

Video presentation: <https://drive.google.com/file/d/1TARreFrYcpibNIpK1Uo6iej6iJCPtoCY/view?usp=sharing>

Language and libraries: Python 3; PyCryptodome (Task 2), PyCA cryptography (Task 3), and the standard library hashlib, hmac and os (Task 4). Full code listings and program outputs are in the Appendix.

Task 1: Theory and Cipher Design

a) Confusion, diffusion and the avalanche effect

Confusion makes the relationship between the encryption key and the ciphertext complex, so it is difficult for an attacker to work out the key from the ciphertext. In AES, **SubBytes** provides confusion by replacing each byte using a non-linear substitution table, the S-box.

Diffusion spreads the effect of one plaintext bit across many ciphertext bits, so patterns in the original message are hard to recognise. In AES, **ShiftRows** and **MixColumns** provide diffusion by rearranging and mixing the data across the state.

The **avalanche effect** means that a small change in the input, such as one plaintext bit, should change many bits of the ciphertext. AES achieves this through repeated rounds of substitution, shifting, mixing and key addition. Ideally, changing one bit changes about half of the ciphertext bits. Task 2 confirms this experimentally.

b) DES vs AES

Property	DES	AES
Block size	64 bits	128 bits
Key size	56 effective bits (64 with parity)	128, 192 or 256 bits

Number of rounds	16	10, 12 or 14 (by key size)
Cipher structure	Feistel network	Substitution-permutation network
Present-day security	Insecure: the 56-bit key is too small and can be brute-forced	Secure when properly implemented with an appropriate key size

c) ECB vs CBC vs CTR

ECB (Electronic Codebook): each plaintext block is encrypted independently, so identical plaintext blocks give identical ciphertext blocks under the same key. This reveals patterns, which makes ECB unsuitable for most applications.

CBC (Cipher Block Chaining): each plaintext block is XORed with the previous ciphertext block before encryption. CBC needs an IV so that identical plaintexts do not produce identical ciphertexts, and it needs padding to fill whole blocks.

CTR (Counter): AES encrypts a nonce and counter to produce a keystream, which is XORed with the plaintext. It needs no padding and can encrypt data of any length.

Purpose of an IV/nonce: it gives each encryption a fresh starting value, so encrypting the same plaintext with the same key does not produce the same ciphertext.

Misuse to avoid: a nonce must **never be reused with the same key in CTR mode**. Reuse produces the same keystream and exposes the XOR of the plaintexts, which seriously weakens security.

Task 2: Python Avalanche Experiment

Using AES-128 with a fixed key (1234567890abcdef), we encrypted the 16-byte block `Sixteen byte txt.` We then flipped one plaintext bit at a time (bits 0 to 9), encrypted again with the same key, and counted the ciphertext bits that changed out of 128.

Trial	Bit flipped	Bits changed (of 128)	% changed
1	0	59	46.09%
2	1	63	49.22%
3	2	59	46.09%
4	3	68	53.12%
5	4	73	57.03%
6	5	64	50.00%
7	6	64	50.00%
8	7	59	46.09%
9	8	66	51.56%
10	9	64	50.00%

Average	63.9	49.92%
---------	------	--------

Interpretation. On average, flipping a single plaintext bit changed **63.9 of 128 bits (49.92%)**, very close to the ideal 50%. Every trial fell between 46.09% and 57.03%, so the behaviour is consistent and no bit position is weak. This is the avalanche effect produced by AES's confusion and diffusion. For security, it means the ciphertexts of two almost identical plaintexts look unrelated, so an attacker learns nothing about how similar the inputs were and cannot recover the key or plaintext by making small changes and watching the output.

Task 3: AES Modes in Practice

We used the PyCA cryptography library with a fresh random AES-256 key. The message was The quick brown fox jumps over the lazy dog!! (45 characters / 45 bytes). CBC used a fresh random 16-byte IV with PKCS7 padding. CTR used a fresh random 16-byte nonce/counter block with no padding. The values below come from the saved run in Appendix A.2. The key, IV and nonce are random, so the hex values differ on every run, but the lengths do not.

	AES-CBC	AES-CTR
IV / nonce (hex)	e67536c42a52961142c0354f800e0cc6	c4bac638200e4d4b33e62df2cd11b754
Plaintext length	45 bytes	45 bytes
Ciphertext length	48 bytes	45 bytes
Ciphertext (hex)	77df3b718be62e6335fc94babd5ad9dcf9b11ffb73a9f04455bab7812d24faa9c311145a022e230348d90e9c53eeb30d	9ef68c78ea5ddacc001329edc75fcb4e7386dd2c36b0186b1ea95adcbd1e3d209bc4e6d0fb7e7873bec32607a8
Recovered plaintext matches exactly?	True	True

The script compares each recovered plaintext with the original using == and stops with an error if either does not match. Both passed.

Observations comparing CBC and CTR

1. **Padding and ciphertext length.** CBC works on whole 16-byte blocks, so the 45-byte message was PKCS7-padded to 48 bytes (3 blocks), and the ciphertext is longer than the plaintext. CTR turns AES into a stream cipher: it XORs the plaintext with a keystream made by encrypting the counter, so it needs no padding and the ciphertext is exactly 45 bytes. Neither mode hides the message length.
2. **Parallelism.** CBC encryption is sequential, because each block is XORed with the previous ciphertext block before encryption (CBC decryption can run in parallel). In CTR every block uses its own counter value, so encryption and decryption can both run in parallel, and any block can be decrypted on its own.
3. **IV/nonce reuse.** Reusing a CTR nonce with the same key repeats the keystream, so XORing two ciphertexts gives the XOR of the two plaintexts. This is catastrophic. Reusing a CBC IV is also

wrong but milder: it reveals when two messages begin with the same blocks. Both modes here use a fresh random value on every encryption.

4. **Error propagation and malleability.** Flipping one bit of a CBC ciphertext block garbles that whole plaintext block and flips the same bit in the next block. In CTR it flips exactly one plaintext bit, so an attacker can make precise changes without knowing the key. Neither mode provides integrity, which motivates Task 4.

Task 4: Integrity and Tampering

We computed a SHA-256 hash and an HMAC-SHA256 tag (with a fresh 32-byte random key) for the original message, then changed one character, fox to box (index 16, f to b), and recomputed both. Values are from the saved run in Appendix A.3.

	Original message	Modified message
Message	The quick brown fox jumps over the lazy dog!!	The quick brown box jumps over the lazy dog!!
SHA-256	e8af66642705ee57fa03c5af5043d78260528448ec429b95e7c2547db1e3716d	a77516c3a66ba61957b0e691cbe3058dc88d393e904f735a3216eb9f7a66cf5a
HMAC-SHA256	6c49e8a73508e7ebb8ae14aa22e03c69225a54759cf06363760a27190631e97b	5ef97473e043fdeb60a0f732449d9e107efe6165c042e6ef78245ab91ba1ed01
HMAC verification against the original tag	PASS	FAIL

How the digest/tag changed. One changed character flipped **142/256 (55.47%)** of the SHA-256 digest bits and **112/256 (43.75%)** of the HMAC tag bits. The outputs look completely unrelated, which is the avalanche effect again, so no one can tell from the outputs how similar the two messages were.

Verification. The receiver, who holds the key, recomputes the tag and compares it in constant time with `hmac.compare_digest`. The original message verifies (**PASS**) and the modified message is rejected (**FAIL**).

Why a plain hash does not authenticate the sender. SHA-256 uses no secret, so anyone can compute it, including an attacker. If a sender transmits the message together with its SHA-256 hash, an attacker in the middle can change the message, recompute the hash and replace it. The receiver's check then passes and the forgery is accepted. Our run shows this: the attacker's recomputed hash passed the receiver's check (*tampering NOT detected*). A plain hash only detects accidental corruption.

Why HMAC provides integrity and authentication. HMAC mixes a secret key into the hash, so only someone who holds the key can produce a valid tag. An attacker without the key cannot compute a matching tag for a changed message. In our run, the attacker's tag failed verification (*tampering detected*). A valid tag therefore shows that the message was not changed (**integrity**) and that it came from a holder of the shared key (**authentication**). HMAC does not give non-repudiation, because both parties share the key. That would need a digital signature.

Task 5: Reflection

The most important security lesson we learned:

The most important lesson we learned is that a strong cipher does not make a system secure on its own. How it is used matters just as much.

Our avalanche experiment showed that AES itself is very strong. Flipping a single plaintext bit changed 49.9% of the ciphertext bits on average, almost exactly the ideal 50%. But Task 3 showed that this strength depends on correct use. AES-CBC and AES-CTR both recovered our message exactly, and each encryption used a fresh random IV or nonce, so the same message never produced the same ciphertext. Reusing a CTR nonce with the same key would reveal the XOR of two plaintexts without anyone breaking AES. We also saw that encryption only hides content. Neither mode detects a modified ciphertext, and CTR even lets an attacker flip chosen plaintext bits.

Task 4 completed the picture. An attacker defeated a plain SHA-256 hash simply by recomputing it, but HMAC-SHA256 rejected the tampered message because the attacker did not have the secret key.

Secure communication therefore needs confidentiality and integrity together, plus careful handling of keys, IVs and nonces. This is why authenticated encryption such as AES-GCM is preferred in practice.

Video presentation: <https://drive.google.com/file/d/1TARreFrYcpibNIpK1Uo6iej6iJCPtoCY/view?usp=sharing>

The video (5 minutes or less) includes all group members, shows each program running, and explains the security meaning of the results.

Appendix: Code and Program Output

To reproduce: `pip install -r requirements.txt`, then run each script from the `block_ciphers_aes_and_des` folder.

A.1 Task 2: `task2_avalanche/avalanche.py`

```
from Crypto.Cipher import AES

# -----
# 1. AES-128 key
# AES-128 requires a 16-byte key
# -----

key = b"1234567890abcdef"

# -----
# 2. 16-byte plaintext
# -----

plaintext = b"Sixteen byte txt"

# Check that the plaintext is exactly 16 bytes
print("Plaintext:", plaintext)
print("Plaintext length:", len(plaintext), "bytes")

# -----
# 3. Encrypt one 16-byte block
# -----

def encrypt_block(data: bytes) -> bytes:
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.encrypt(data)

# -----
# 4. Flip exactly one bit
# -----

def flip_bit(data: bytes, bit_index: int) -> bytes:
    b = bytearray(data)

    byte_i, offset = divmod(bit_index, 8)

    b[byte_i] ^= (1 << offset)

    return bytes(b)

# -----
# 5. Count how many ciphertext bits changed
# -----

def count_changed_bits(x: bytes, y: bytes) -> int:
    return sum(
        (a ^ b).bit_count()
        for a, b in zip(x, y)
    )
```

```

# -----
# 6. Perform one avalanche trial
# -----

def avalanche_trial(encrypt_block, plaintext: bytes, bit_index: int):

    # Encrypt the original plaintext
    c1 = encrypt_block(plaintext)

    # Flip exactly one bit and encrypt again
    c2 = encrypt_block(
        flip_bit(plaintext, bit_index)
    )

    # Count changed ciphertext bits
    return count_changed_bits(c1, c2)

# -----
# 7. Encrypt the original plaintext
# -----

original_ciphertext = encrypt_block(plaintext)

print("\nOriginal ciphertext:")
print(original_ciphertext.hex())

# -----
# 8. Run at least 10 different single-bit flips
# -----

bit_positions = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

results = []

for bit_index in bit_positions:

    changed_bits = avalanche_trial(
        encrypt_block,
        plaintext,
        bit_index
    )

    percentage = (changed_bits / 128) * 100

    results.append(
        (bit_index, changed_bits, percentage)
    )

# -----
# 9. Display the results
# -----

print("\nAES-128 Avalanche Experiment")
print("-" * 50)

print(
    f"{'Bit Flipped':<15}"
    f"{'Changed Bits':<15}"
    f"{'Percentage':<15}"
)

for bit_index, changed_bits, percentage in results:

    print(

```

```

        f"{bit_index:<15}"
        f"{changed_bits:<15}"
        f"{percentage:.2f}%"
    )

# -----
# 10. Calculate the average
# -----

average_changed_bits = (
    sum(result[1] for result in results)
    / len(results)
)

average_percentage = (
    average_changed_bits / 128
) * 100

print("\nAverage changed ciphertext bits:",
      round(average_changed_bits, 2))

print("Average percentage of changed ciphertext bits:",
      round(average_percentage, 2), "%")

```

Output (results/task2_avalanche_output.txt):

```

Plaintext: b'Sixteen byte txt'
Plaintext length: 16 bytes

Original ciphertext:
9b3f58d7dd6a907062594e39ce8d0451

AES-128 Avalanche Experiment
-----

```

Bit Flipped	Changed Bits	Percentage
0	59	46.09%
1	63	49.22%
2	59	46.09%
3	68	53.12%
4	73	57.03%
5	64	50.00%
6	64	50.00%
7	59	46.09%
8	66	51.56%
9	64	50.00%

```

Average changed ciphertext bits: 63.9
Average percentage of changed ciphertext bits: 49.92 %

```

A.2 Task 3: task3_aes_modes/aes_modes.py

```

# Task 3: AES Modes in Practice (CBC and CTR)

import os

from cryptography.hazmat.primitives import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

BLOCK_SIZE = 16 # AES block size in bytes (128 bits)

# Message of at least 40 characters (45 bytes, so CBC padding is visible)

```

```

MESSAGE = b"The quick brown fox jumps over the lazy dog!!"

def aes_cbc_encrypt(key: bytes, plaintext: bytes):
    # Fresh random IV for every encryption
    iv = os.urandom(BLOCK_SIZE)

    # PKCS7 pad the plaintext to a multiple of the block size
    padder = padding.PKCS7(BLOCK_SIZE * 8).padder()
    padded = padder.update(plaintext) + padder.finalize()

    # Encrypt with AES-CBC
    encryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).encryptor()
    ciphertext = encryptor.update(padded) + encryptor.finalize()

    return iv, ciphertext

def aes_cbc_decrypt(key: bytes, iv: bytes, ciphertext: bytes):
    # Decrypt with AES-CBC using the same IV
    decryptor = Cipher(algorithms.AES(key), modes.CBC(iv)).decryptor()
    padded = decryptor.update(ciphertext) + decryptor.finalize()

    # Remove the PKCS7 padding
    unpadder = padding.PKCS7(BLOCK_SIZE * 8).unpadder()
    plaintext = unpadder.update(padded) + unpadder.finalize()

    return plaintext

def aes_ctr_roundtrip(key: bytes, plaintext: bytes):
    # Fresh random 16-byte initial counter block (nonce + counter)
    nonce = os.urandom(BLOCK_SIZE)

    # Encrypt with AES-CTR (no padding needed, it acts as a stream cipher)
    encryptor = Cipher(algorithms.AES(key), modes.CTR(nonce)).encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()

    # Decrypt with AES-CTR using the same nonce
    decryptor = Cipher(algorithms.AES(key), modes.CTR(nonce)).decryptor()
    recovered = decryptor.update(ciphertext) + decryptor.finalize()

    return nonce, ciphertext, recovered

if __name__ == "__main__":
    # AES-256 key
    key = os.urandom(32)

    print("=" * 60)
    print("Task 3: AES Modes in Practice")
    print("=" * 60)
    print(f"Message           : {MESSAGE.decode()}")
    print(f"Message length    : {len(MESSAGE)} bytes")
    print(f"Key (hex)         : {key.hex()}")

    # CBC encrypt + decrypt
    iv, cbc_ct = aes_cbc_encrypt(key, MESSAGE)
    cbc_pt = aes_cbc_decrypt(key, iv, cbc_ct)
    cbc_ok = cbc_pt == MESSAGE

    print("\n--- AES-CBC ---")
    print(f"IV (hex)           : {iv.hex()}")
    print(f"Ciphertext (hex)  : {cbc_ct.hex()}")
    print(f"Ciphertext length : {len(cbc_ct)} bytes")
    print(f"Recovered         : {cbc_pt.decode()}")
    print(f"Exact match       : {cbc_ok}")

```

```

# CTR encrypt + decrypt
nonce, ctr_ct, ctr_pt = aes_ctr_roundtrip(key, MESSAGE)
ctr_ok = ctr_pt == MESSAGE

print("\n--- AES-CTR ---")
print(f"Nonce (hex)      : {nonce.hex()}")
print(f"Ciphertext (hex)  : {ctr_ct.hex()}")
print(f"Ciphertext length : {len(ctr_ct)} bytes")
print(f"Recovered          : {ctr_pt.decode()}")
print(f"Exact match        : {ctr_ok}")

# Verify both recovered plaintexts exactly match the original
assert cbc_ok, "CBC round trip failed"
assert ctr_ok, "CTR round trip failed"

print("\n--- Summary ---")
print(f"CBC ciphertext length: {len(cbc_ct)} bytes "
      f"(padded from {len(MESSAGE)} to a multiple of {BLOCK_SIZE})")
print(f"CTR ciphertext length: {len(ctr_ct)} bytes "
      f"(same as plaintext, no padding)")
print("Both plaintexts recovered exactly: PASS")

```

Output (results/task3_aes_modes_output.txt):

```

=====
Task 3: AES Modes in Practice
=====
Message      : The quick brown fox jumps over the lazy dog!!
Message length : 45 bytes
Key (hex)    : c846c6c8150d7115b7a405df49f529afb64f79e908f83dd176bebedb5c2b802c

--- AES-CBC ---
IV (hex)      : e67536c42a52961142c0354f800e0cc6
Ciphertext (hex) : 77df3b718be62e6335fc94babd5ad9dcf9b11ffb73a9f04455bab7812d24faa9c311145a022e230348d9
0e9c53eeb30d
Ciphertext length : 48 bytes
Recovered       : The quick brown fox jumps over the lazy dog!!
Exact match     : True

--- AES-CTR ---
Nonce (hex)    : c4bac638200e4d4b33e62df2cd11b754
Ciphertext (hex) : 9ef68c78ea5ddacc001329edc75fcb4e7386dd2c36b0186b1ea95adcbd1e3d209bc4e6d0fb7e7873bec3
2607a8
Ciphertext length : 45 bytes
Recovered       : The quick brown fox jumps over the lazy dog!!
Exact match     : True

--- Summary ---
CBC ciphertext length: 48 bytes (padded from 45 to a multiple of 16)
CTR ciphertext length: 45 bytes (same as plaintext, no padding)
Both plaintexts recovered exactly: PASS

```

A.3 Task 4: task4_integrity/integrity.py

```

# Task 4: Integrity and Tampering (SHA-256 and HMAC-SHA256)
# Owner: Member 3

import hashlib
import hmac
import os

# Same message as Task 3
MESSAGE = b"The quick brown fox jumps over the lazy dog!!"

# Tampered copy with exactly ONE character changed ("fox" -> "box")

```

```

TAMPER_INDEX = MESSAGE.index(b"fox")
MODIFIED = MESSAGE[:TAMPER_INDEX] + b"b" + MESSAGE[TAMPER_INDEX + 1:]

def sha256_digest(message: bytes) -> str:
    return hashlib.sha256(message).hexdigest()

def make_hmac(key: bytes, message: bytes) -> bytes:
    return hmac.new(key, message, hashlib.sha256).digest()

def verify_hmac(key: bytes, message: bytes, tag: bytes) -> bool:
    # Recompute the tag and compare in constant time
    expected = make_hmac(key, message)
    return hmac.compare_digest(expected, tag)

def count_changed_bits(x: bytes, y: bytes) -> int:
    return sum((a ^ b).bit_count() for a, b in zip(x, y))

if __name__ == "__main__":
    # Secret key shared only by the sender and the receiver
    key = os.urandom(32)

    print("=" * 60)
    print("Task 4: Integrity and Tampering")
    print("=" * 60)
    print(f"Original message : {MESSAGE.decode()}")
    print(f"Modified message : {MODIFIED.decode()}")
    print(f"Changed position : index {TAMPER_INDEX} "
          f"('{chr(MESSAGE[TAMPER_INDEX])}' -> '{chr(MODIFIED[TAMPER_INDEX])}')"")
    print(f"HMAC key (hex) : {key.hex()}")

    # Digest and tag of the original message
    orig_hash = sha256_digest(MESSAGE)
    orig_tag = make_hmac(key, MESSAGE)

    # Digest and tag of the modified message
    mod_hash = sha256_digest(MODIFIED)
    mod_tag = make_hmac(key, MODIFIED)

    # How the digest and tag change after tampering
    hash_bits = count_changed_bits(bytes.fromhex(orig_hash), bytes.fromhex(mod_hash))
    tag_bits = count_changed_bits(orig_tag, mod_tag)

    print("\n--- SHA-256 ---")
    print(f"Original : {orig_hash}")
    print(f"Modified : {mod_hash}")
    print(f"Bits changed : {hash_bits}/256 ({hash_bits / 256:.2%})")

    print("\n--- HMAC-SHA256 ---")
    print(f"Original tag : {orig_tag.hex()}")
    print(f"Modified tag : {mod_tag.hex()}")
    print(f"Bits changed : {tag_bits}/256 ({tag_bits / 256:.2%})")

    # Receiver checks the original message against the original tag
    ok_original = verify_hmac(key, MESSAGE, orig_tag)
    # Receiver checks the tampered message against the original tag
    ok_modified = verify_hmac(key, MODIFIED, orig_tag)

    print("\n--- HMAC verification (receiver holds the key) ---")
    print(f"Original message + original tag : {'PASS' if ok_original else 'FAIL'}")
    print(f"Modified message + original tag : {'PASS' if ok_modified else 'FAIL'}")

    # Why a plain hash does not authenticate the sender:
    # an attacker who changes the message can simply recompute SHA-256,

```

```

# because SHA-256 needs no secret. The receiver's hash check then passes.
forged_hash = sha256_digest(MODIFIED)
# Receiver hashes what arrived and compares it with the attached hash
hash_check = sha256_digest(MODIFIED) == forged_hash

# The attacker cannot do the same with HMAC: without the secret key,
# any tag they make (here, with a guessed key) fails verification.
attacker_key = os.urandom(32)
forged_tag = make_hmac(attacker_key, MODIFIED)
forged_ok = verify_hmac(key, MODIFIED, forged_tag)

print("\n--- Attacker modifies the message and replaces the checksum ---")
print(f"Attacker's SHA-256 : {forged_hash}")
print(f"Receiver hash check : {'PASS (tampering NOT detected)' if hash_check else 'FAIL'}")
print(f"Attacker's HMAC tag : {forged_tag.hex()}")
print(f"Receiver HMAC check : {'PASS' if forged_ok else 'FAIL (tampering detected)'}")

assert orig_hash != mod_hash, "SHA-256 digest did not change"
assert orig_tag != mod_tag, "HMAC tag did not change"
assert ok_original, "HMAC should verify for the original message"
assert not ok_modified, "HMAC should fail for the modified message"
assert hash_check, "Recomputed plain hash should match"
assert not forged_ok, "Forged HMAC should fail without the key"

print("\n--- Summary ---")
print("One changed character changes about half the bits of both outputs.")
print("HMAC verifies the original message and rejects the modified one.")
print("A plain hash can be recomputed by anyone, so it cannot prove who")
print("sent the message. HMAC needs the secret key, so only a key holder")
print("can produce a valid tag: it gives integrity AND authentication.")

```

Output (results/task4_integrity_output.txt):

```

=====
Task 4: Integrity and Tampering
=====
Original message : The quick brown fox jumps over the lazy dog!!
Modified message : The quick brown box jumps over the lazy dog!!
Changed position : index 16 ('f' -> 'b')
HMAC key (hex) : acd5ba8c6cc5268497a711b4495687ba2dce83004e32accb9e19b37b41264008

--- SHA-256 ---
Original : e8af66642705ee57fa03c5af5043d78260528448ec429b95e7c2547db1e3716d
Modified : a77516c3a66ba61957b0e691cbe3058dc88d393e904f735a3216eb9f7a66cf5a
Bits changed : 142/256 (55.47%)

--- HMAC-SHA256 ---
Original tag : 6c49e8a73508e7ebb8ae14aa22e03c69225a54759cf06363760a27190631e97b
Modified tag : 5ef97473e043fdeb60a0f732449d9e107efe6165c042e6ef78245ab91ba1ed01
Bits changed : 112/256 (43.75%)

--- HMAC verification (receiver holds the key) ---
Original message + original tag : PASS
Modified message + original tag : FAIL

--- Attacker modifies the message and replaces the checksum ---
Attacker's SHA-256 : a77516c3a66ba61957b0e691cbe3058dc88d393e904f735a3216eb9f7a66cf5a
Receiver hash check : PASS (tampering NOT detected)
Attacker's HMAC tag : 78030f5b61be0a814821241bd43b4f18a1a583039104eeb9911729cc8821cba5
Receiver HMAC check : FAIL (tampering detected)

--- Summary ---
One changed character changes about half the bits of both outputs.
HMAC verifies the original message and rejects the modified one.
A plain hash can be recomputed by anyone, so it cannot prove who

```

sent the message. HMAC needs the secret key, so only a key holder can produce a valid tag: it gives integrity AND authentication.